

Project - High Level Design

TravelCrew – Multi-Agent Hospitality System

Course Name: Agentic AI

Institution Name: Medicaps University – Datagami Skill Based Course

Sr no	Student Name	Enrolment Number
1	HARSH RAI	EN22CS301393
2	HEMANT DHAKAD	EN22CS301421
3	GAURAV DWIVEDI	EN23CS3L1008
4	HIMANI JAISWAL	EN22CS30123
5	HARSHWARDHAN YADAV	EN22CS301416
6	GOUTAM VERMA	EN22CS301375

Group Name: 12D4

Project Number: AAI- 08

Industry Mentor Name:

University Mentor Name: Pradeep Baniya

Academic Year: 2025 - 2026

Table of Contents

1. Introduction.
 - 1.1. Scope of the document.
 - 1.2. Intended Audience
 - 1.3. System overview.
2. System Design.
 - 2.1. Application Design
 - 2.2. Process Flow.
 - 2.3. Information Flow.
 - 2.4. Components Design
 - 2.5. Key Design Considerations
 - 2.6. API Catalogue.
3. Data Design.
 - 3.1. Data Model
 - 3.2. Data Access Mechanism
 - 3.3. Data Retention Policies
 - 3.4. Data Migration
4. Interfaces
5. State and Session Management
6. Caching
7. Non-Functional Requirements
 - 7.1. Security Aspects
 - 7.2. Performance Aspects
8. References

1. Introduction

TravelCrew is an Agentic AI-based system developed to provide intelligent, personalized travel planning using a collaborative multi-agent architecture. The system integrates a Large Language Model (LLM) with a Retrieval-Augmented Generation (RAG) knowledge base and a live web search tool to transform a natural language travel request into a complete, structured travel itinerary.

The system is built on the CrewAI framework and demonstrates advanced inter-agent communication, sequential hand-off protocols, and orchestrated multi-tool workflows. Two specialized agents — a Researcher Agent and a Writer Agent — collaborate to research destinations and produce high-quality, personalized travel plans.

The system enables users to specify destination, duration, travel type, budget, interests, and special requirements through a professional Streamlit web interface, receiving a complete day-by-day itinerary as output.

1.1 Scope of the Document

This document provides the High-Level Design (HLD) of the TravelCrew — Multi-Agent Hospitality System. It includes:

1. Overall system architecture and design philosophy
2. Multi-agent orchestration design
3. RAG pipeline and knowledge base design
4. Data design and information flow
5. API integrations and external service dependencies
6. Security, performance, and non-functional requirements

This document serves as a technical reference for implementation, evaluation, and future enhancement of the system.

1.2 Intended Audience

The intended audience for this document includes:

- Developers and Software Engineers
- AI Engineers
- Academic Evaluators
- Industry Mentors
- System Architects

1.3 System Overview

TravelCrew accepts a structured travel request from the user through a Streamlit web interface. The system workflow includes:

1. User submits travel preferences via the Streamlit UI (destination, days, travel type, budget, interests, special requirements).
2. The system constructs a rich natural language travel request from user inputs.
3. CrewAI kicks off the sequential multi-agent workflow.
4. The Researcher Agent queries the ChromaDB knowledge base using semantic search (RAG), then supplements with live web search via Serper API.
5. The research report is handed off to the Writer Agent via CrewAI's context parameter.
6. The Writer Agent synthesizes a complete day-by-day travel itinerary with hotel recommendations, dining, attractions, budget breakdown, and travel tips.
7. The final itinerary is displayed to the user through the Streamlit UI.

The system is modular, API-driven, and demonstrates advanced agentic AI principles including specialization, hand-off protocols, and state management.

2. System Design

The system follows a layered modular architecture.

Layers include:

- Presentation Layer (Frontend UI)
- Application Layer (Business Logic)
- AI Services Layer (LLM + Image Generation APIs)
- Data Layer (Vector Database)

The design ensures scalability, modularity, and ease of integration.

2.1 Application Design

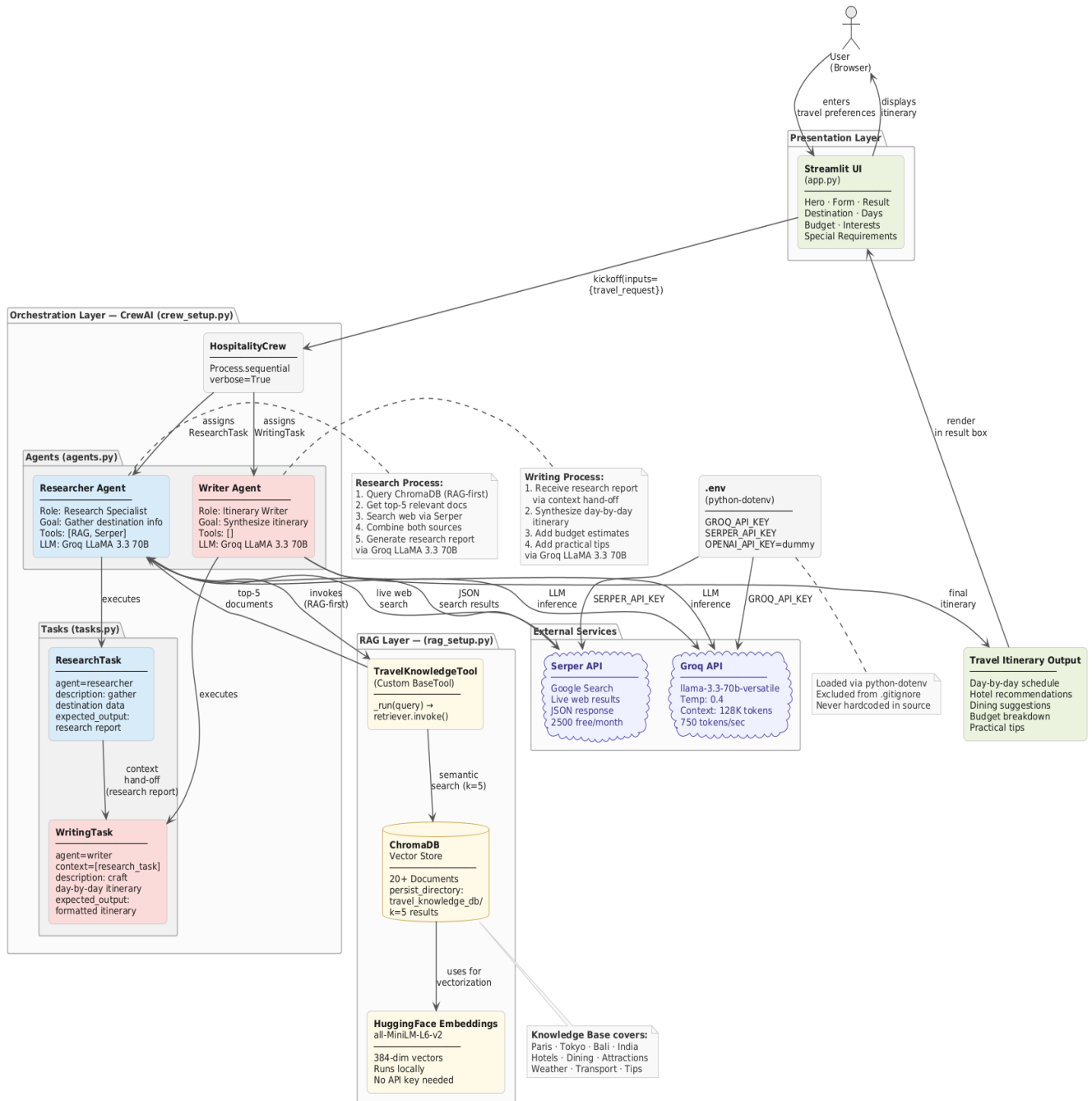
Frontend (app.py):

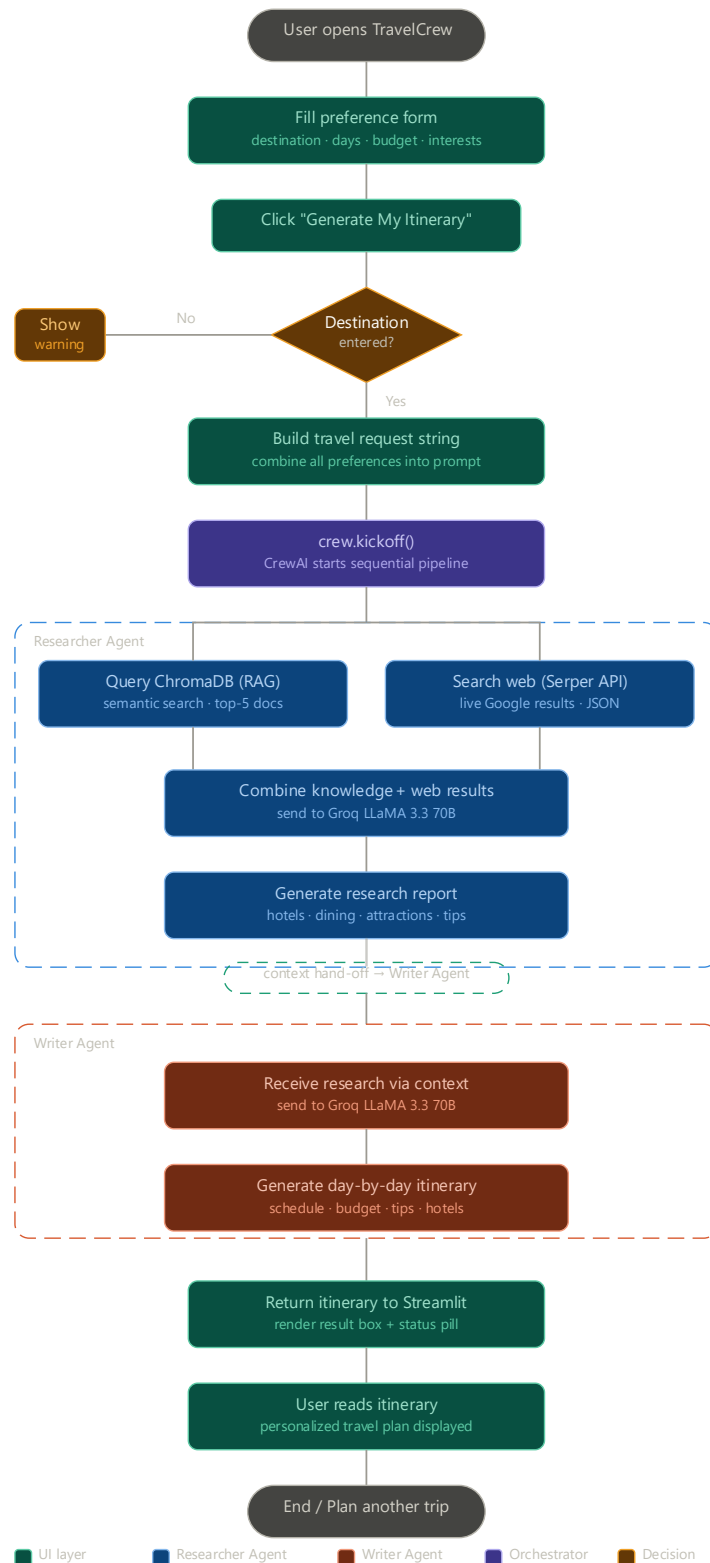
- Built using Streamlit with a cinematic dark theme using Unbounded and Inter fonts
- Full-screen hero section with navigation bar, scrolling marquee, and agent showcase
- Structured preference form: destination, days slider, travel type, budget range, interests multiselect, special requirements
- Live trip summary pills showing all selections before submission
- Responsive design with mobile breakpoints

Backend (agents.py, tasks.py, crew_setup.py, rag_setup.py):

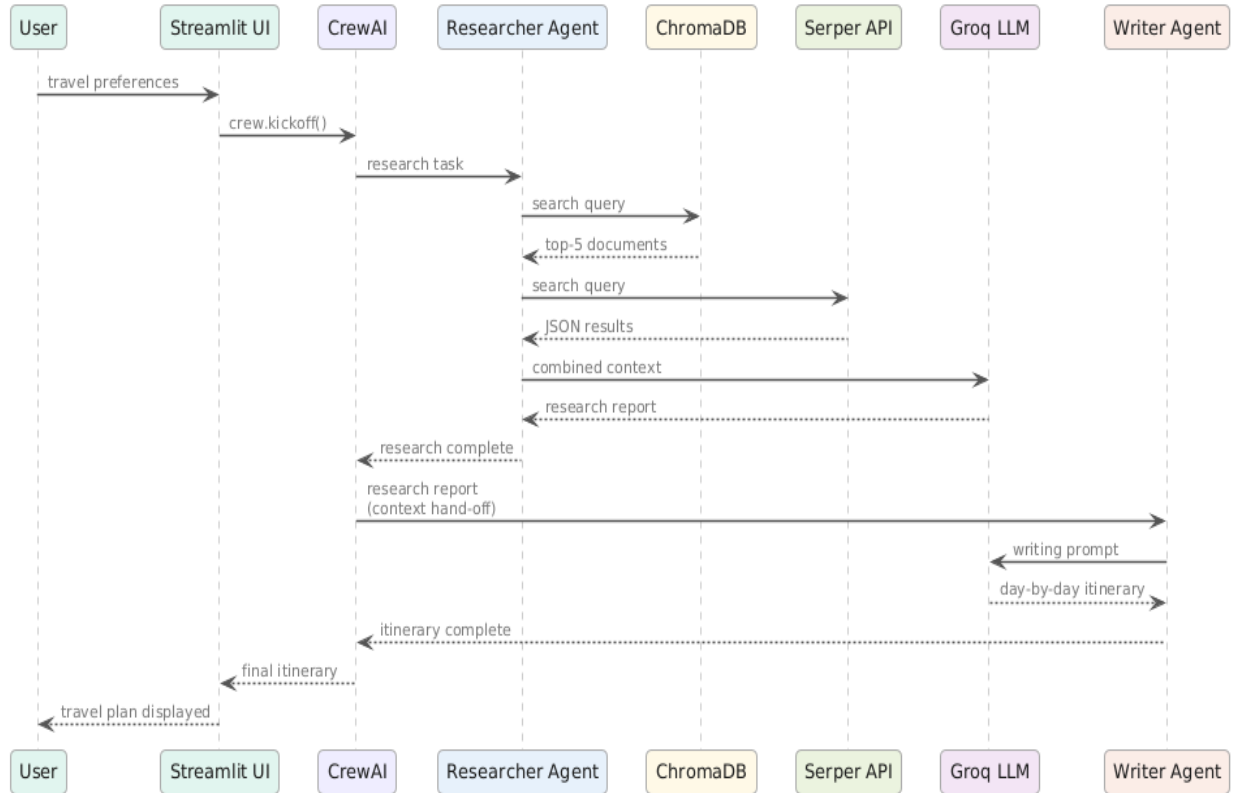
- Python-based processing using CrewAI framework
- Manages agent definitions, task descriptions, tool integrations, and crew orchestration
- Handles API communication with Groq LLM and Serper Search
- Manages ChromaDB vector store initialization, persistence, and retrieval

TravelCrew — Multi-Agent Hospitality System Architecture



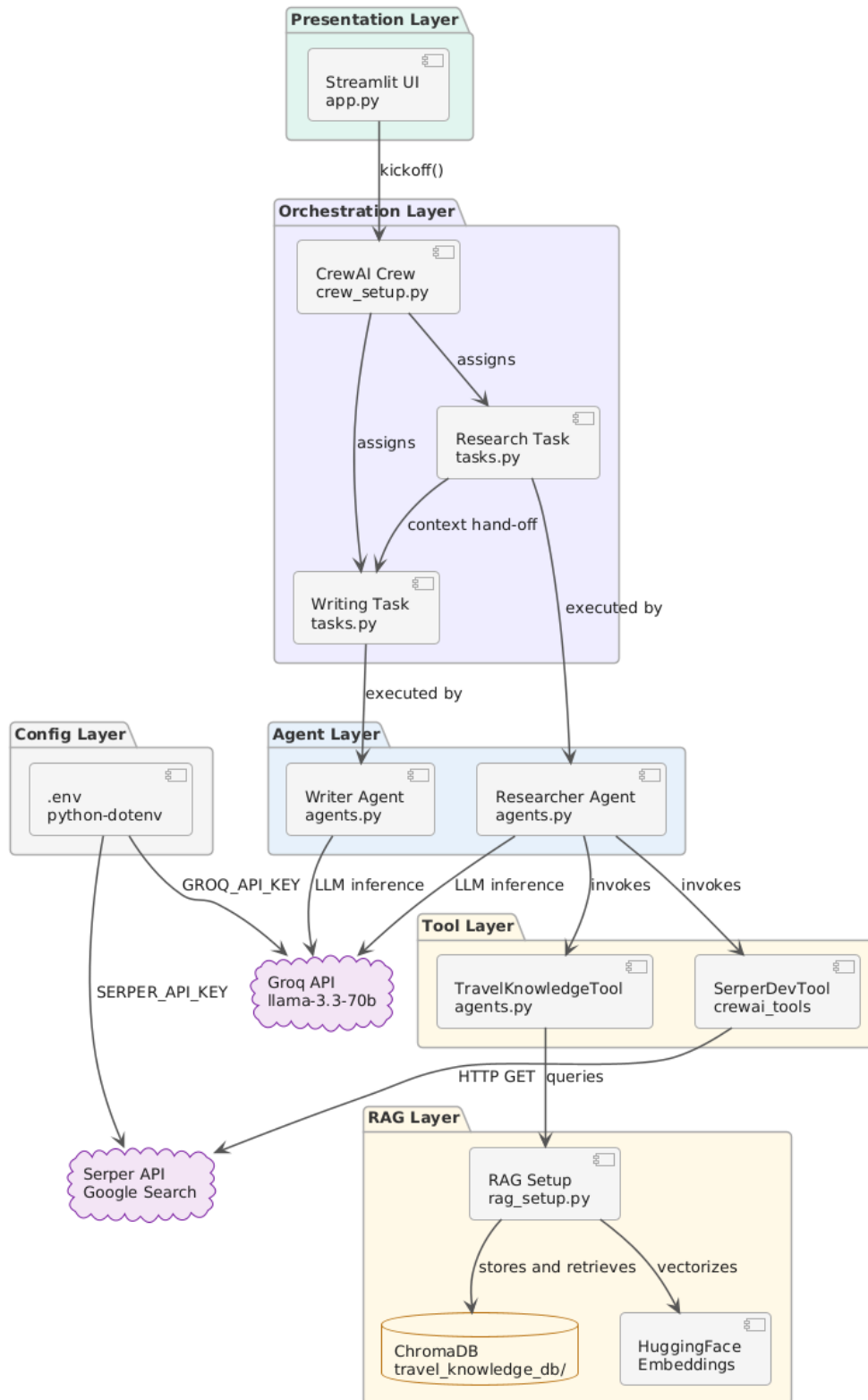


TravelCrew - Information Flow Diagram



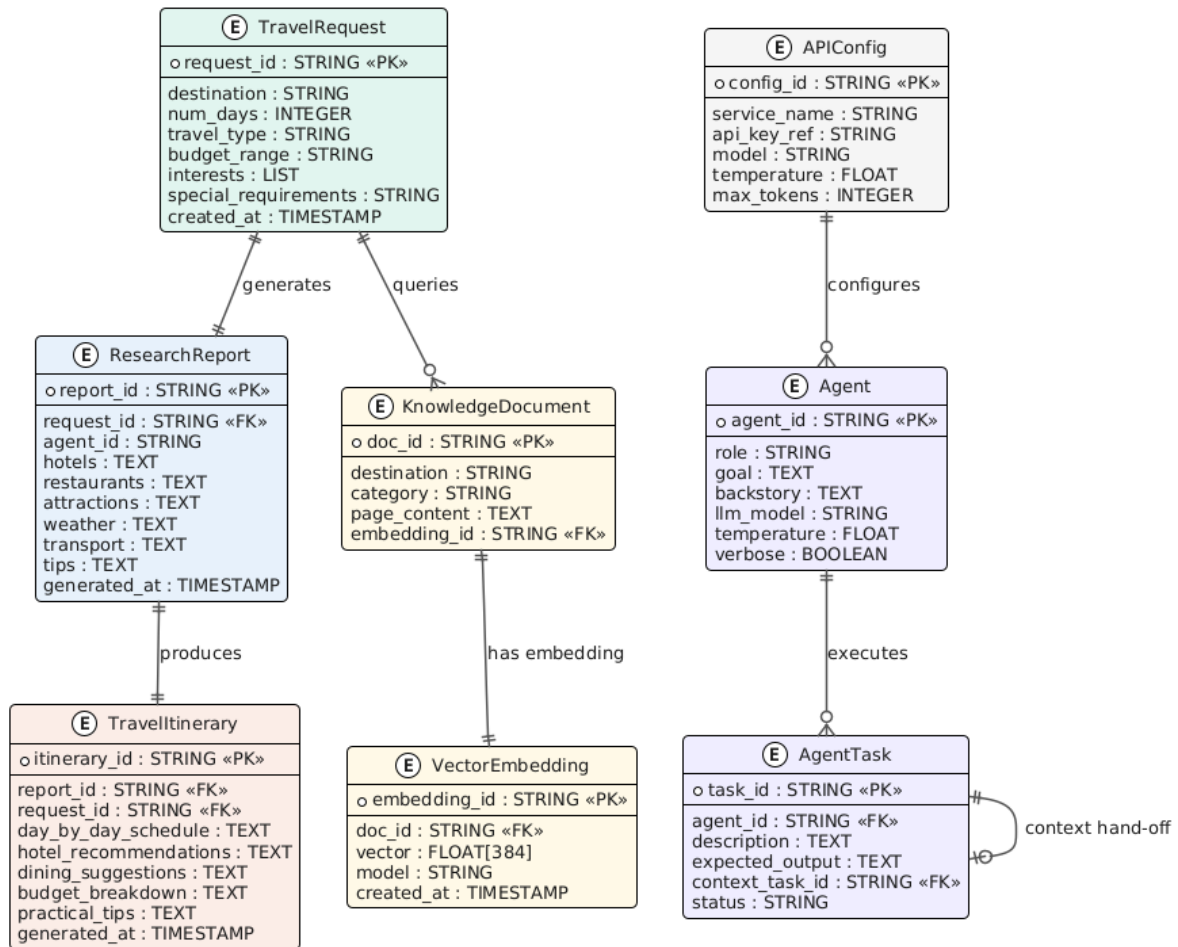
2.4 Component Design

TravelCrew - Component Diagram



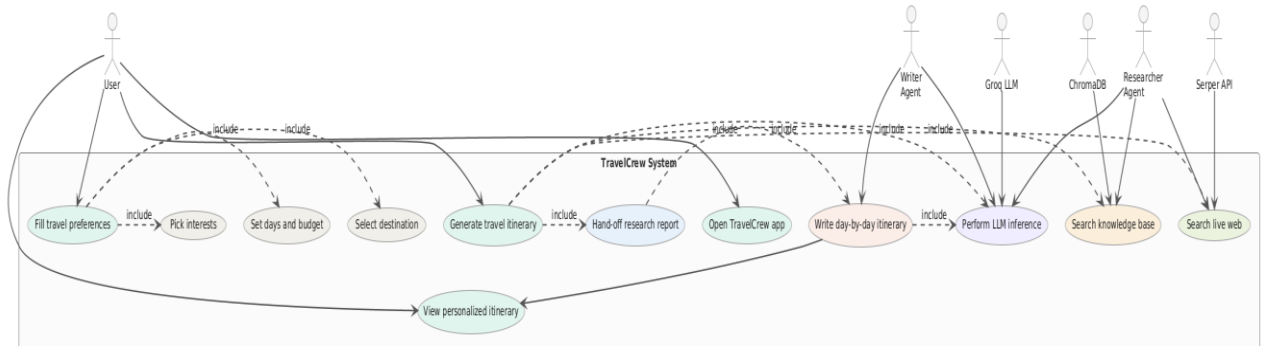
2.5 ER Design:

TravelCrew - ER Diagram



2.6 Use Case Diagram

TravelCrew - Use Case Diagram



2.7 Key Design Considerations

1. **Agent Specialization:** Each agent has a distinct role, goal, and backstory ensuring optimal task performance without overlap.
2. **RAG-First Strategy:** Researcher queries ChromaDB before web search to reduce API calls and improve grounded responses.
3. **Modular Architecture:** Each component is independent and replaceable without affecting other components.
4. **Secure API Management:** All API keys stored in .env file, loaded via python-dotenv, excluded from version control.
5. **Large Context Window:** Groq LLaMA 3.3 70B provides 128K token context ensuring large research reports pass fully between agents.
6. **Future Extensibility:** Architecture supports adding more agents (e.g. Budget Agent, Weather Agent) without redesign.

2.8 API Catalogue

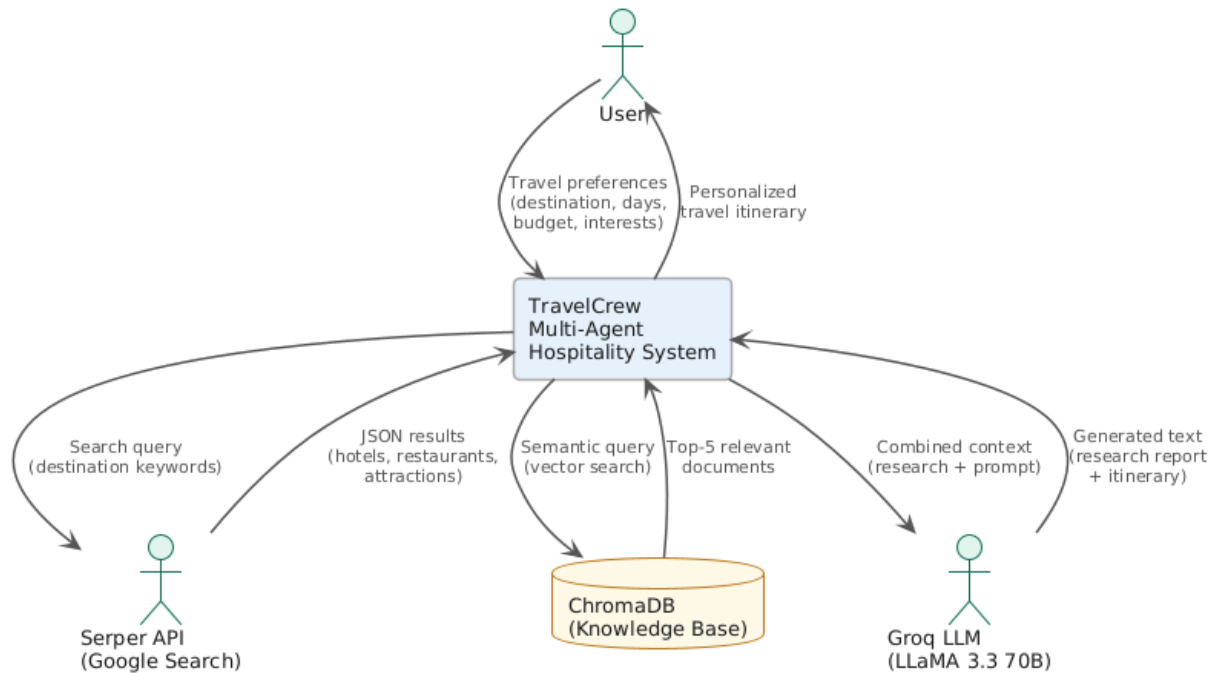
API / Service	Provider	Purpose	Details
Groq Inference API	Groq	LLM inference for both agents	Model: llama-3.3-70b-versatile, Temp: 0.4, Context: 128K tokens
Serper Search API	Serper.dev	Live Google Search	Returns JSON search results. Free tier: 2,500 searches/month
HuggingFace Embeddings	HuggingFace	Text vectorization	Model: all-MiniLM-L6-v2, runs locally, no API key required
ChromaDB	Chroma Research	Vector store (local)	Persisted in travel_knowledge_db/; auto-persists on first run

3. Data Design

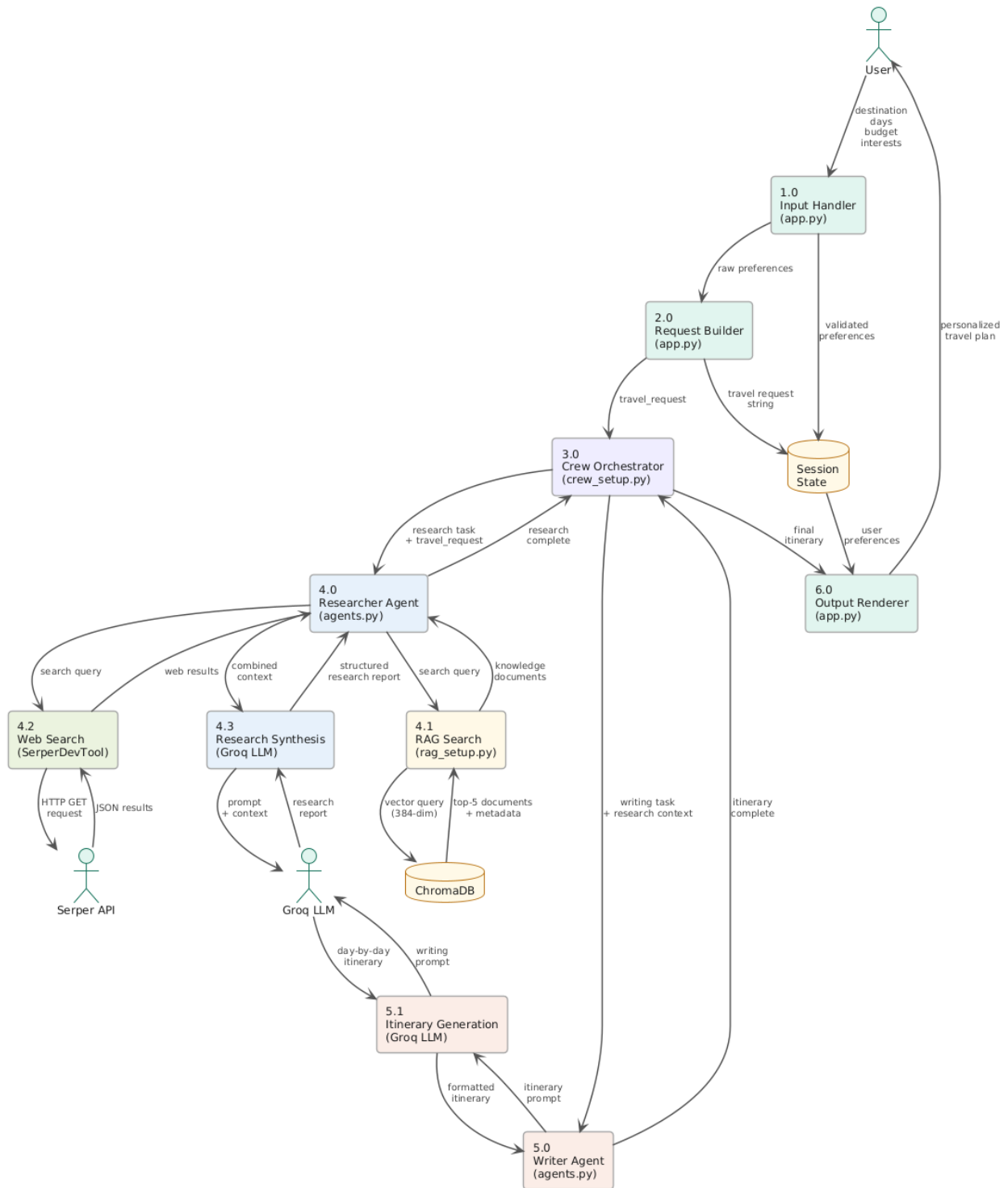
- TravelCrew manages the following types of data throughout its pipeline:
- User travel preferences (destination, days, travel type, budget, interests, special requirements)
- Constructed travel request strings
- ChromaDB knowledge documents with metadata (destination, category)
- Text embeddings (384-dimension vectors from all-MiniLM-L6-v2).
- Agent research reports and synthesized itinerary outputs.

3.1 Data Model

TravelCrew - Data Flow Diagram (Level 0 - Context Diagram)



TravelCrew - Data Flow Diagram (Level 2)



3.2 Data Access Mechanism

- **REST API Calls:** The system communicates with external AI services (LLM API and Image Generation API) using REST APIs to send prompts and receive responses.
- **JSON-Based Communication:** Data exchange between frontend, backend, and APIs uses JSON format for structured and efficient request–response handling.
- **Token-Based Authentication:** Secure API access is maintained using authentication tokens stored as environment variables.
- **HTTPS Communication:** All data transmission occurs over HTTPS to ensure encrypted and secure communication

3.3 Data Retention Policies

- **Vector Store Persistence:** ChromaDB embeddings are persisted locally in the travel_knowledge_db/ directory across application restarts.
- **Session Data:** User preferences and generated itineraries are stored in Streamlit session state and cleared on browser refresh.
- **Data Privacy:** No sensitive user information is permanently stored. API keys are excluded from version control via .gitignore

3.4 Data Migration

Future scalability options include:

- **Cloud Migration:** Supports migration to cloud-based vector databases for scalability.
- **Model Upgrades:** Architecture allows easy replacement or upgrade of AI models.
- **Schema Evolution:** Data structure can be extended without affecting existing functionality

4. Interfaces

4.1 User Interface

- Web-based interface built with Streamlit, accessible at <http://localhost:8501>
- Cinematic dark theme with Unbounded and Inter typography, responsive across desktop and mobile
- Structured multi-section preference form replacing a single text box for richer, personalized input

4.2 External AI Interfaces

- **Groq LLM API:** Used by both agents for reasoning, research synthesis, and itinerary writing via LiteLLM bridge.
- **Serper Search API:** Used by Researcher Agent for live web search of current destination data.

4.3 Data Interface

- ChromaDB Interface: Local vector store accessed via LangChain Chroma wrapper. Retriever returns top-5 semantically similar documents per query.
- CrewAI Context Interface: Internal data passing mechanism that transfers the Researcher's output to the Writer via the context parameter.

5. State and Session Management

1. Managed using Streamlit session state (`st.session_state`) for active user sessions.
2. Temporarily stores user preferences, constructed travel request, and generated itinerary during session.
3. CrewAI internally maintains agent task state ensuring research data is preserved during the Researcher-to-Writer hand-off.
4. Session resets upon application refresh. Future enhancement: persistent itinerary history using JSON file storage.

6. Caching

- **ChromaDB Embedding Cache:** Knowledge base embeddings are computed once on first run and persisted to disk. All subsequent runs load from cache, eliminating re-computation cost.
- **RAG Result Reuse:** Semantically similar queries retrieve cached ChromaDB documents, reducing Serper API calls for known destinations.
- **Streamlit Caching:** `st.cache_resource` can be applied to the ChromaDB initialization function to prevent re-loading on every Streamlit rerun.
- **Overall Benefit:** Reduces latency, minimizes external API calls, and improves system responsiveness for repeat or similar queries.

7. Non-Functional Requirements

TravelCrew must satisfy the following non-functional requirements:

- **Scalability:** Modular agent-based design supports adding new agents, tools, and destinations without full system redesign.
- **Reliability:** Stable operation with graceful error handling for API timeouts, invalid inputs, and missing API keys.
- **Maintainability:** Clean modular file structure (`agents.py`, `tasks.py`, `crew_setup.py`, `rag_setup.py`, `app.py`) enables easy updates.
- **Usability:** Professional, responsive Streamlit UI with structured form inputs requiring no technical knowledge from users.
- **Security:** All credentials managed via environment variables, excluded from version control, transmitted over HTTPS.
- **Performance:** Groq provides up to 750 tokens/sec inference; ChromaDB local retrieval is sub-second; system targets sub-60 second end-to-end itinerary generation.

7.1 Security Aspects

- API keys (GROQ_API_KEY, SERPER_API_KEY) stored securely in .env file, loaded via python-dotenv at runtime.
- All external API communication secured via HTTPS protocol (Groq API, Serper API).
- .env file excluded from version control via .gitignore to prevent accidental credential exposure.
- Input validation on user preferences to prevent malformed or excessively long inputs.
- ChromaDB runs locally with no external network exposure, ensuring knowledge base data privacy.

7.2 Performance Aspects

- Groq provides ultra-fast LLM inference (up to 750 tokens/second) minimizing agent processing time.
- ChromaDB local semantic search executes in milliseconds, avoiding network latency for knowledge retrieval.
- RAG-first strategy reduces the number of Serper API calls for well-known destinations covered in ChromaDB.
- LLaMA 3.3 70B's 128K context window handles large research reports without truncation or information loss.
- HuggingFace embeddings run fully locally with no API dependency, contributing zero network latency to the RAG pipeline.

8. References

- [1] CrewAI Inc., "CrewAI Documentation," *CrewAI*, 2024. [Online]. Available: <https://docs.crewai.com>. [Accessed: Mar. 2026].
- [2] Groq Inc., "Groq API Documentation," *Groq*, 2024. [Online]. Available: <https://console.groq.com/docs>. [Accessed: Mar. 2026].
- [3] Chroma Research Inc., "ChromaDB Documentation," *Chroma*, 2024. [Online]. Available: <https://docs.trychroma.com>. [Accessed: Mar. 2026].
- [4] LangChain Inc., "LangChain Documentation," *LangChain*, 2024. [Online]. Available: <https://python.langchain.com>. [Accessed: Mar. 2026].
- [5] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *Proc. EMNLP*, 2019. [Online]. Available: <https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>. [Accessed: Mar. 2026].
- [6] Serper Technologies, "Serper API Documentation," *Serper.dev*, 2024. [Online]. Available: <https://serper.dev/docs>. [Accessed: Mar. 2026].
- [7] Streamlit Inc., "Streamlit Documentation," *Streamlit*, 2024. [Online]. Available: <https://docs.streamlit.io>. [Accessed: Mar. 2026].
- [8] Python Software Foundation, "Python 3 Official Documentation," *Python.org*, 2024. [Online]. Available: <https://docs.python.org/3/>. [Accessed: Mar. 2026].